Abstract Data Types (ADTs)

- An abstract data type (ADT) is an abstraction of a data structure.
- An ADT specifies:
  - Data stored
  - Operations on the data
  - Error conditions associated with operations

Example: ADT modeling a simple stock trading system

- The data stored are buy/sell orders
- The operations supported are
  - order buy(stock, shares, price)
  - order sell(stock, shares, price)
  - void cancel(order)
- Error conditions:
  - Buy/sell a nonexistent stock
  - Cancel a nonexistent order

While studying an ADT

You should understand

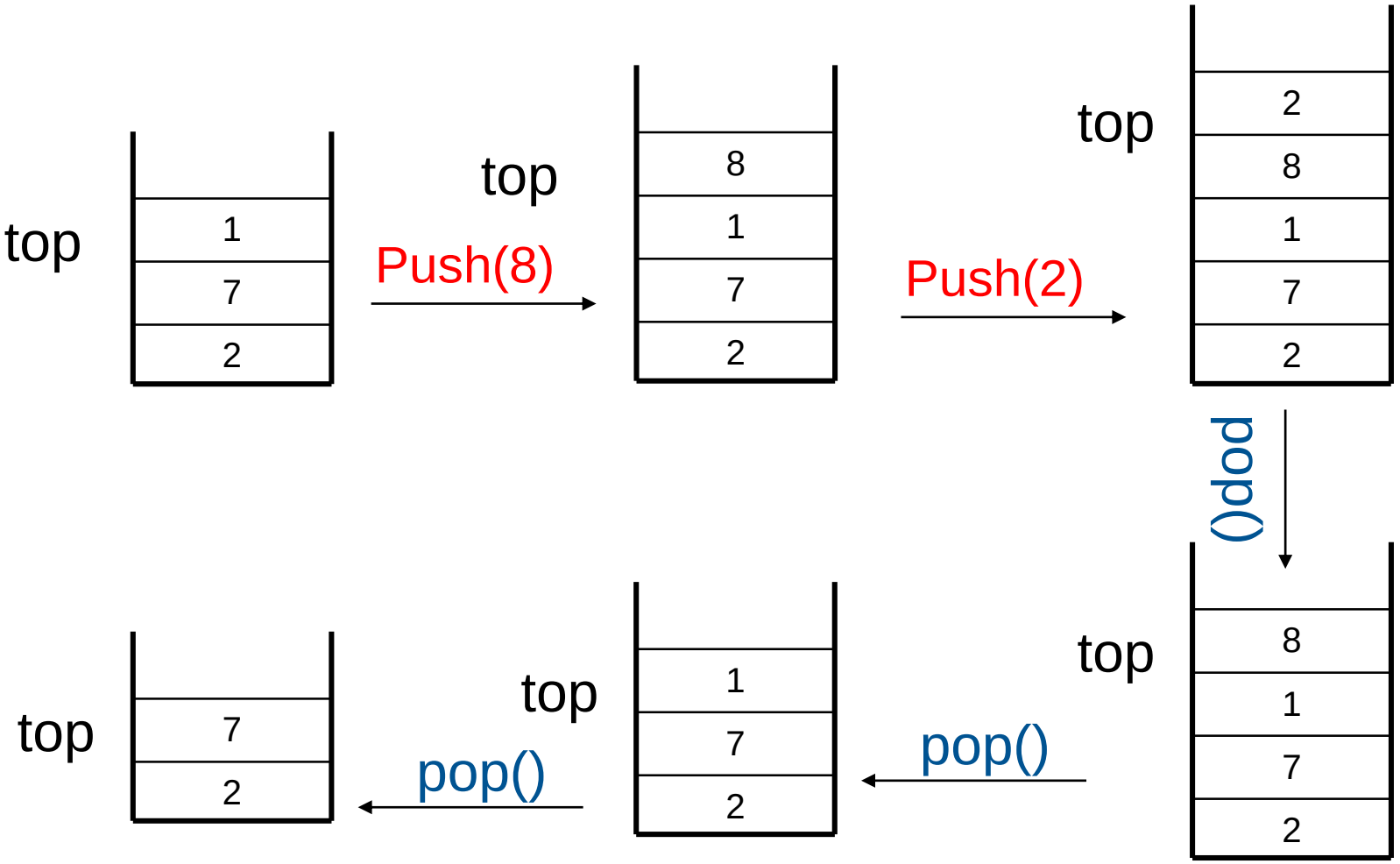
- kind of data (type, domain)
- behaviour of data (functions defined on data)
  - primary function and auxiliary function
- algorithm (how function works)

Stack



- Stack is a data structure in which data is added and removed at only one end called the top.
- To *insert* (i.e. push) an item to the stack, it must be placed on the top of the stack.
- To *delete* (i.e. pop) an item from the stack, it must be removed from the top of the stack.
- Thus, the last element that is pushed into the stack is the first element to be popped out of the stack.
  - i.e., stack is a Last In First Out (LIFO) data structure

An example of a stack



Applications of Stacks

- Direct applications
  - Page-visited history in a Web browser
  - Undo sequence in a text editor
  - Saving local variables when one function calls another, and this one calls another, and so on.
  - Chain of method calls in the runtime environment

- Indirect applications
  - Auxiliary data structure for algorithms

The Stack ADT

- The Stack ADT stores arbitrary objects.
- Insertions and deletions follow the last-in first-out scheme.
- Main stack operations:
  - push(object o): inserts element o
  - pop(): removes and returns the last inserted element

- Auxiliary stack operations:
  - top(): returns a reference to the last inserted element without removing it
  - size(): returns the number of elements stored
  - isEmpty(): returns a Boolean value indicating whether no elements are stored
  - isFull(): ??

Exceptions

- Attempting the execution of an operation of ADT may sometimes cause an error condition, called an exception.
- Exceptions are said to be "thrown" by an operation that cannot be executed.

- In the Stack ADT, operations pop and top cannot be performed if the stack is empty.
- Attempting the execution of pop or top on an empty stack should throw an Exception

Implementation of Stacks

Stack can be implemented by:

- Arrays (static: the size of stack is given initially)
- Linked lists (dynamic: the size of stack varies)

Array Implementation

Need to declare an array size ahead of time.

- for an empty stack, set Top to -1

Push

(1) Increment Top by 1

(2) Set Stack[Top] = X

Pop

(1) Set return value to Stack[Top]

(2) Decrement Top by 1

These operations are performed in constant time.

Array-based Stack

- A simple way of implementing the Stack ADT uses an **array**.
- We add elements from left to right.
- A variable **t** keeps track of the index of the top element.

```
Algorithm pop():
if isEmpty() then
        throw Exception
else
        t = t – 1
        return S[t + 1]
```

```
Algorithm push(o)
if t = S.length – 1 then
        throw Exception
else
        t = t + 1
        S[t] ← o
```



Growable Array-based Stack

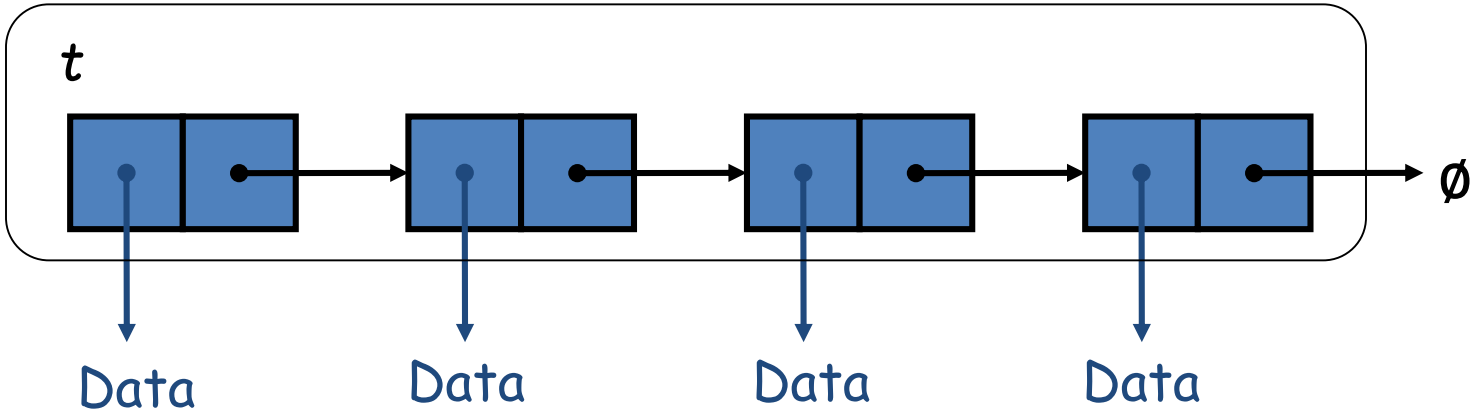
- In a push operation, when the array is full, instead of throwing an exception, we can replace the array with a larger one.

- How large should the new array be?
  - incremental strategy: increase the size by a constant c
  - doubling strategy: double the size

```
Algorithm push(o)
if t = S.length − 1 then
        A ← new array of size
        for i ← 0 to t do
                A[i] ← S[i]
                S ← A
        t ← t + 1
        S[t] ← o
```

Stack with a List

- We can implement a stack with a linked list.
- The top element is pointed at the first node of the list.
- The space used is $O(n)$ and each operation of the Stack ADT takes $O(1)$ time.



Case study 1 - Evaluating Postfix Expression

(5+9)*2+6*5

- An ordinary arithmetical expression like the above is called infix-expression (i.e. binary operators appear between their operands)

- The order of operations evaluation is determined by the precedence rules.

- When an evaluation order is desired that is different from that provided by the precedence, parentheses are used to override precedence rules.

Case study 1 - Evaluating Postfix Expression

- Expressions can also be represented using postfix notation (i.e. each operator comes after its two operands).

- The advantage of postfix notation is that the order of operation evaluation is *unique* without the need for precedence rules or parenthesis.

Infix	**Postfix**
(a + b)* c	a b + c *
2 + 14 * 5	2 14 5 * +
(6 – 2) * (5 + 4)	6 2 - 5 4 + *

Case study 1 - Evaluating Postfix Expression

The following algorithm uses a stack to evaluate a postfix expression.

```
Start with an empty stack
for (each item in the expression) {
          if (the item is a number)
               Push the number onto the stack
          else if (the item is an operator){
               Pop two operands from the stack
                   Apply the operator to the operands
               Push the result onto the stack
            }
}
Pop the only one number from the stack – that's the result
of the evaluation
```

Case study 2 : Balancing Symbols

- To check that every right brace, bracket, and parentheses must correspond to its left counterpart.

- Algorithm
  (1) Make an empty stack.
  (2) Read characters until end of file
      i. If the character is an opening symbol, push it onto the stack
      ii. If it is a closing symbol and if the stack is empty, report an error
      iii. Otherwise, pop off the stack. If the symbol popped out is not the corresponding opening symbol, then report an error
  (3) At end of file, if the stack is not empty, report an error